Part I: First principles

Results

We created a new model and did two update steps, using a learning rate of 0.01 for the LinearModel and 0.2 for the LinearSigmoidModel. The target is t = (0.25; 0.75). Figures 1 and 2 show the gradients of the second update. For the learning rate comparison (Figure 3, Table 1) we trained for 10 steps, each time from the same starting weights.

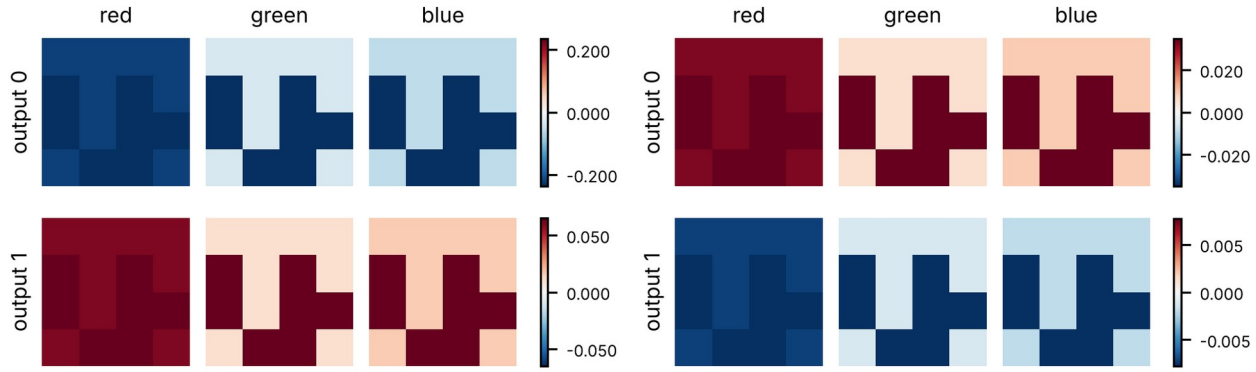


Figure 1 (left), Figure 2 (right): gradients of the second update step for the LinearModel and the LinearSigmoidModel.

Table 1: MSE of the LinearSigmoidModel at step 10 (start: 0.0274).

lr	0.0001	0.001	0.01	0.1	1
MSE	0.0273	0.0265	0.0198	0.0013	≈ 0

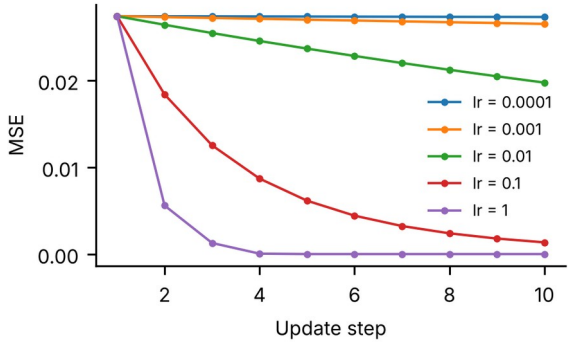


Figure 3: training loss of the LinearSigmoidModel for five learning rates.

Explanation

LinearModel: Before the second update our model gave y = (0.0135; 0.8150), so the first output was still far too low and the second one slightly too high. The gradient of a weight is (y − t)·x, so −0.2365 for output 0 and +0.0650 for output 1. The update pushes the weights of output 0 up, while the weights of output 1 go down. The white pixels equal 1 in all three channels, so they receive the full error. The red pixels are bright in the red channel but dark in green and blue. That is why the red maps look almost uniform, whereas you can clearly recognise the logo in the green and blue maps.

LinearSigmoidModel: This model gave y = (0.3956; 0.7122), so here it is the other way around: output 0 is too high and output 1 too low, which explains the reversed signs. The sigmoid adds one extra factor to the gradient, (y − t)·y(1 − y)·x. We still see the same pattern as in Figure 1, since x remains the only part that differs per weight. The values are clearly smaller though, because y(1 − y) can never exceed 0.25.

Learning rate: The loss drops for all five learning rates, and the larger the learning rate, the faster it drops. With 0.0001 and 0.001 almost nothing happens in 10 steps, whereas with 1 the output already reaches the target after a few steps.

Part II: Linear regression

Results

We trained the model with L1Loss and SGD for 25 epochs. Table 2 and Figure 4 compare the learning rates and batch sizes on 5 000 houses. We then repeated the training with the chosen combination (lr = 0.25, batch size 64) for each dataset size, always using the same 20 test houses (Table 3, Figure 5). The real function is f(x) = 5 000·x + 100 000.

Table 2: results for 5 000 houses after 25 epochs. Plateau: first epoch within 1 % of the lowest loss of all runs.

lr	Batch	Final L1 [€]	Plateau [epoch]
0.01	32	252 622	not reached
0.1	32	43 997	12
0.25	32	44 000	6
0.01	64	299 281	not reached
0.1	64	44 112	23
0.25	**64**	**43 996**	**10**

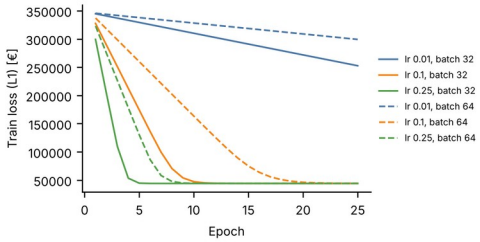


Figure 4: training loss per combination (solid: batch 32, dashed: batch 64).

Table 3: lr = 0.25, batch size 64, 25 epochs.

Samples	Updates	Learned g(x)	Train RMSE [€]	Test RMSE [€]
1 000	400	4 963.70·x + 99.26	110 271	125 721
2 000	800	6 877.77·x + 143.19	53 858	66 510
5 000	1 975	6 911.06·x + 166.11	55 123	66 220
10 000	3 925	6 896.77·x + 204.63	54 931	66 324

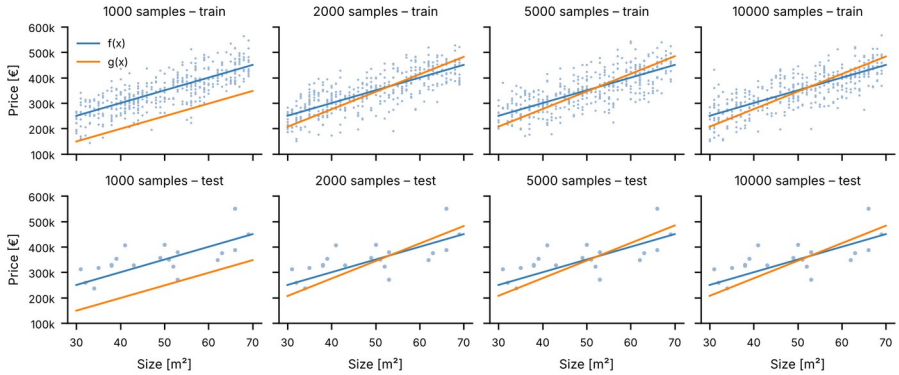


Figure 5: real function f(x) and learned function g(x) on the training set (top, at most 400 points shown) and the test set (bottom).

Explanation

Learning rate and batch size: The larger the learning rate, the sooner the loss reaches its plateau. With 0.01 the model simply did not have enough time: it still has not converged after 25 epochs. Batch size 64 gives fewer updates per epoch than batch size 32 (79 instead of 157), so it needs about twice as many epochs. We compared the final losses and selected lr = 0.25 with batch size 64, which has the lowest value. The difference with batch size 32 is only €3, so both work well, and batch 32 is even faster.

Selected model: The RMSE is €55 123 on the training set and €66 220 on the test set. Since the data contains €50 000 of noise, most of this error cannot be avoided. The test error is higher, which is partly because the test set only has 20 houses. Looking at Figure 5, the line goes through the middle of the data but is too steep and starts too low.

Dataset size: From 2 000 samples on, the results barely change. Extra data does not help here, since a model with only two parameters cannot overfit. The model with 1 000 samples is clearly worse on both sets, but that is underfitting: at batch size 64 it only gets 400 updates, so training stops before the plateau is reached.

Part III: Image classification

Results

Table 4: NeuralNetworkClassificationModel.

Layer	Type	Input → output	Activation
Flatten	–	1×28×28 → 784	–
layer1	Linear	784 → 392	ReLU
layer2	Linear	392 → 196	ReLU
layer3	Linear	196 → 10	none

Table 5: learning rate sweep, batch size 64, 10 epochs.

lr	Train loss	Validation loss	Best validation accuracy
0.001	1.1208	1.0338	78.71 %
0.01	0.2475	0.2460	92.82 %
0.1	**0.0292**	**0.0704**	**97.92 %**

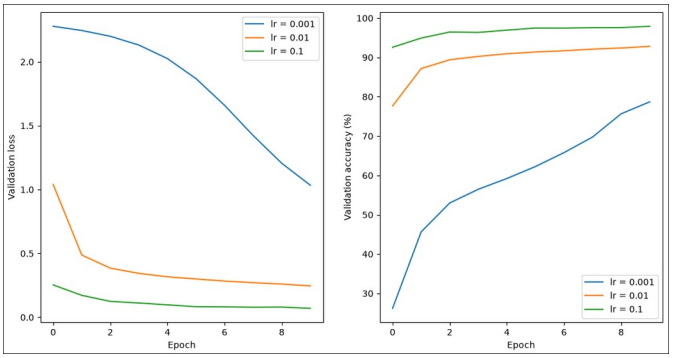


Figure 6: validation loss (left) and validation accuracy (right) per learning rate, batch size 64.

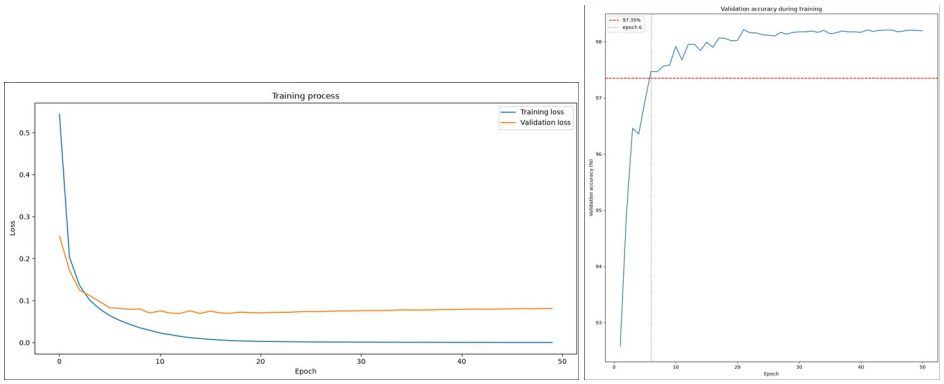


Figure 7 (left): training and validation loss for 50 epochs with lr = 0.1. Figure 8 (right): validation accuracy with the 97.35 % target.

Explanation

Architecture: We flatten the 28×28 image to 784 inputs and use one output per digit. In between are two hidden layers, each half the size of the previous one. We did not tune these sizes any further. For the activation we chose ReLU, because it does not shrink the gradients as much as a sigmoid does. As optimizer we used SGD.

Learning rate. Out of the three values we tried, 0.1 gave the lowest validation loss and the highest accuracy, so we continued with that one.

Training for 50 epochs: The training loss keeps falling towards zero, but the validation loss reaches its lowest point at epoch 15 (0.0695, accuracy 97.99 %) and then slowly rises again, so from there on the model starts to overfit. The highest accuracy, 98.22 %, was reached at epoch 21.

Epochs to reach 97.35 %: We first reached the target after 6 epochs (97.47 %, train loss 0.0642, validation loss 0.0831). Most of the progress happens in those first epochs: the other 44 epochs added far less.